

Patuakhali Science and Technology University
Faculty of Computer Science and Engineering

CIT 324 :: Simulation and Modeling Sessional

Project Report



CloudCalc

FLOW

Project Title : CloudCalc Flow

Submission Date : 11 July 2026

Submitted from,	Submitted to,
Md. Sharafat Karim ID : 2102024, Reg : 10151, Semester : 6 (Level-3, Semester-2)	1. Md Mahbubur Rahman Associate Professor, Department of Computer Science and Information Technology, Patuakhali Science and Technology University. 2. Dr. Md Abdul Masud Professor, Department of Computer Science and Information Technology, Patuakhali Science and Technology University.

Contents

1. Introduction	3
2. Objectives	3
3. Problem Statement	3
4. Literature Review	4
5. Related Commercial Projects	4
6. Scope	4
6.1. Job Market Analysis	5
7. Methodology	5
7.1. Technology Stack	5
7.2. Design Principles	5
8. Simulation Model	6
9. Feature Comparison	7
10. Visual Models	7
10.1. System Architecture	7
10.2. Simulation Flow	8
10.3. Data Flow	8
10.4. Connection Rules	9
10.5. Timeline (Gantt Chart)	9
11. User Interface	10
12. Security and Privacy	11
13. Future Plans	11
14. Result	11
15. Conclusion	12
16. References	13



CloudCalc Flow

Visual Cloud Architecture Simulator and Cost Estimator

1. Introduction

CloudCalc Flow is a fully client-side, visual cloud architecture simulator and cost estimator. It lets a user drag infrastructure components — compute servers, load balancers, API gateways, SQL and NoSQL databases, object storage, in-memory cache, a CDN and a message queue — onto an interactive canvas, wire them together under enforced connection rules, and then simulate a variable concurrent-user load. As the load changes, the application recomputes each server's RAM utilisation, response time and throughput, aggregates a live monthly cost, flags overloaded (out-of-memory) nodes and proposes concrete remediation.

Unlike spreadsheet-style cloud pricing calculators, CloudCalc Flow couples **topology** with **behaviour**: the same diagram that documents an architecture also drives a lightweight performance-and-cost model, so a student or engineer can reason about capacity and budget together, entirely in the browser and without any account, backend or data leaving the device.

2. Objectives

1. To build a responsive, node-based canvas for composing cloud architectures through drag-and-drop with enforced, type-safe connection rules.
2. To implement a transparent, real-time simulation of RAM utilisation, response time and throughput as a function of the simulated concurrent-user load.
3. To estimate the total monthly cost of an architecture live, and to detect overloaded servers and surface actionable upgrade/scale/reduce suggestions with their cost deltas.
4. To provide full undo/redo, localStorage autosave, JSON export/import and a copy-as-text summary — all client-side.
5. To keep the domain logic pure and unit-tested, and the interface accessible, themeable (light/dark) and responsive.

3. Problem Statement

Planning a cloud deployment forces two questions to be answered at once: **what will it look like** and **what will it cost and sustain**. Existing tools tend to answer only one. Native cloud pricing calculators (AWS, Google Cloud, Azure) produce accurate bills but present a flat list of line items with no topology and no notion of load-driven capacity. Diagramming tools (draw.io, Lucidchart) capture topology beautifully but know nothing about cost or performance. Visual cost tools such as Cloudcraft bridge some of this gap but are provider-locked, require an account, and keep the model opaque.

The result is that early-stage capacity and budget reasoning — “will one Micro server survive 2,000 users, and what does the next step up cost?” — is done by hand or across disconnected tools. There is a need for a single, instant, no-login, provider-agnostic tool that visualises an architecture and simulates its cost and load-behaviour side by side, with a model simple and transparent enough to teach with.

4. Literature Review

The relationship between server utilisation and responsiveness is a classical result of queueing theory. Kleinrock [1] shows that for an M/M/1-style service, mean response time rises sharply and non-linearly as utilisation approaches 100%, which motivates CloudCalc Flow’s response-time penalty that grows with RAM utilisation and its throughput degradation term near saturation. Little’s Law [2] — that the mean number in a system equals arrival rate times mean time in system — underpins the intuition that throughput and concurrency cannot both grow without bound on fixed resources.

For capacity planning specifically, Menascé and Almeida [3] formalise metrics-driven models that map workload intensity (here, concurrent users) onto resource demand and service level, which is the exact shape of the per-server calculation used in this project. At the platform level, Armbrust et al. [4] frame elasticity and pay-as-you-go economics as the defining properties of cloud computing, and note that the ability to add or remove capacity in response to load is what makes cost a first-class design variable — echoed in CloudCalc Flow’s “add another server” and “upgrade tier” suggestions and their explicit cost deltas.

Finally, the practice of treating cloud spend as an engineering concern is codified by the FinOps Foundation [5], whose framework argues that engineers should see the cost consequences of architectural choices in real time. CloudCalc Flow is a small, educational embodiment of that principle.

5. Related Commercial Projects

1. **AWS Pricing Calculator** [6] gives authoritative, up-to-date pricing for real AWS services, but produces a flat itemised estimate with no architecture diagram and no load or performance simulation.
2. **Google Cloud** [7] and **Azure** [8] calculators are equivalent in spirit — accurate, provider-specific, list-based — and likewise omit topology and capacity behaviour.
3. **Cloudcraft** [9] is the closest visual analogue: it renders live AWS/Azure diagrams with attached cost. However, it is provider-locked, requires an account, is not open source, and does not simulate user-driven load against per-node capacity.
4. **React Flow** / **xyflow** [10] is not a competitor but the enabling library — a node-based editor toolkit — on which CloudCalc Flow’s canvas, custom nodes, edges and connection validation are built.

6. Scope

CloudCalc Flow is delivered as a modern web application built with Next.js and React Flow. It is entirely client-side: there is no server, database or authentication, and no data ever

leaves the browser. The application ships nine component types with a hardcoded but clearly-documented cost and capacity catalogue, a strict connection-rule engine, a pure simulation module, undo/redo history, localStorage persistence, JSON export/import and a markdown summary.

The performance model is intentionally simplified and pedagogical rather than a byte-accurate emulation of any specific cloud provider; its constants (server tiers, OS overheads, per-stack memory footprints and component prices) are transparent and editable in a single data file. The interface is responsive down to mobile widths, supports a light/dark theme, and is keyboard-navigable.

6.1. Job Market Analysis

The stack chosen for CloudCalc Flow — Next.js, React, TypeScript and Tailwind CSS — sits at the centre of current full-stack web-developer demand in Bangladesh and globally. Listings on major portals (BDJobs, Arc.dev, Wellfound) for full-stack and front-end roles consistently name React/Next.js, TypeScript and a utility CSS framework as core requirements, frequently alongside state-management and data-visualisation experience. The competencies exercised by this project — component-driven UI, client-side state architecture, canvas/graph interaction and a self-contained computational model — map directly onto those hiring signals, making CloudCalc Flow a demonstrable portfolio artefact.

7. Methodology

7.1. Technology Stack

Development followed an iterative, agile approach. The stack was selected for performance, type-safety and a fully client-side deployment.

Component	Technology
Frontend Framework	Next.js 16 (React 19, App Router, Turbopack)
Language	TypeScript (strict)
Canvas / Graph	React Flow v12 (@xyflow/react)
State Management	Zustand 5 with immer + persist middleware
Styling	Tailwind CSS v4
Icons	Lucide React
Persistence	Browser localStorage (autosave) + JSON export/import
Testing	Vitest (pure simulation unit tests)
Diagrams	Mermaid; report typeset in Typst

Table 1: Technology stack for CloudCalc Flow

7.2. Design Principles

1. **Pure, testable domain logic.** All cost and performance math lives in dependency-free functions (`lib/simulation.ts`) so it can be unit-tested and reused by both the nodes and the metrics panel.
2. **Single source of truth.** A Zustand store holds nodes, edges and the user-count; every view derives from it, and history snapshots wrap each mutation for undo/redo.

3. **Client-first and private.** No backend, no login; the whole architecture autosaves to localStorage and can be exported as a portable JSON file.
4. **Type-safety end to end.** Discriminated-union node data and a typed connection-rule table make invalid states hard to represent.

8. Simulation Model

The core of CloudCalc Flow is a pure calculation run whenever the user-count slider or any node configuration changes. For each Compute Server in the traffic path:

Quantity	Definition
Base RAM	OS overhead + framework idle
Traffic RAM	$(\text{users} / N_{\text{servers}}) / 100 \times \text{stackPer100}$
Utilisation	$(\text{Base} + \text{Traffic}) / \text{TierRAM} \times 100\%$
Response time	$\text{TierBaseline} + \text{Util} / 100 \times 50\text{ms}$
Throughput	$\text{TierMax} \times (1 - \text{Util} / 150)$

Table 2: Per-server simulation formulas

Here N_{servers} is the number of servers sharing traffic. If any Load Balancer is present, only servers wired from it receive traffic and share the load evenly; otherwise every server receives the full load directly. A server crosses into **Warning** at 80% utilisation and **Overloaded** (OOM) at 100%, at which point its border turns red and remediation suggestions are generated. The response-time penalty and throughput degradation deliberately mirror the qualitative behaviour of a saturating queue [1].

Global metrics aggregate these per-server results: total monthly cost (sum of all node costs), total RAM, traffic-weighted average response time, peak throughput (capped by load-balancer capacity), estimated bandwidth, and the worst node status. The catalogue that feeds the model is summarised below.

Tier	RAM	vCPU	Cost/mo	Max req/s
Micro	1 GB	2	\$8	100
Small	2 GB	2	\$16	500
Medium	4 GB	4	\$32	2000
Large	8 GB	8	\$64	5000

Table 3: Compute server tiers

Component	Cost/mo	Stack (RAM / 100 users)	idle
Load Balancer	\$20	Next.js — 150 MB	120 MB
API Gateway	\$15	Express.js — 80 MB	60 MB
SQL Database	\$8–10 +/-GB	Flask — 50 MB	40 MB
NoSQL Database	\$12–15 +/-GB	Go — 20 MB	15 MB
S3 Storage	\$5 + \$0.10/GB	OS: Ubuntu 400 MB	
Redis Cache	\$15	OS: Alpine 50 MB	

CDN	\$20 + \$0.08/GB	OS: Arch 100 MB	
Message Queue	\$25		

Table 4: Component costs and memory footprints (documented defaults)

9. Feature Comparison

Table 5 contrasts CloudCalc Flow with widely used cloud pricing and diagramming tools.

Tool	Visual canvas	Live cost	Traffic sim	Perf metrics	Client-side	No lo-gin	Export	FOSS
Cloud-Calc Flow	✓	✓	✓	✓	✓	✓	✓	✓
AWS Pricing Calc.	—	✓	—	—	—	✓	✓	—
GCP Calculator	—	✓	—	—	—	✓	✓	—
Azure Calculator	—	✓	—	—	—	✓	✓	—
Cloud-craft	✓	✓	—	—	—	—	✓	—
draw.io	✓	—	—	—	✓	✓	✓	✓

Table 5: Feature comparison of CloudCalc Flow with existing tools

10. Visual Models

10.1. System Architecture

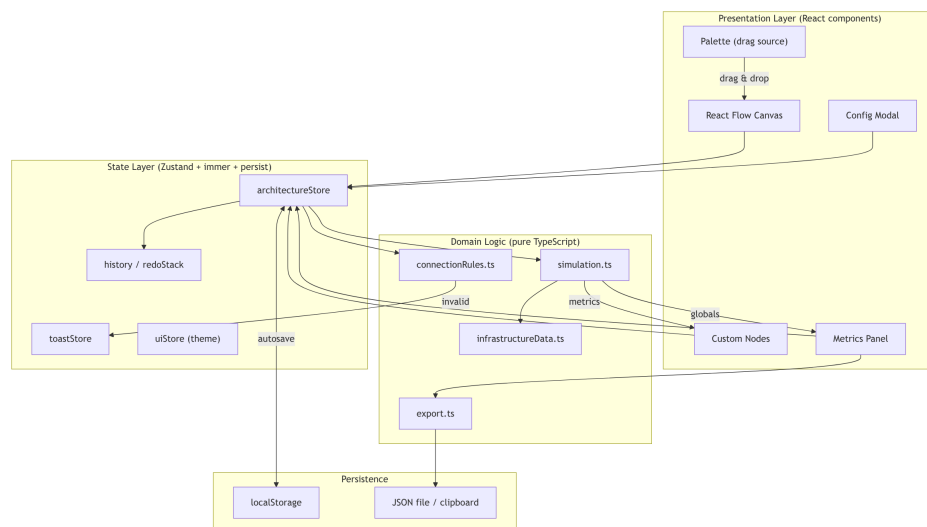


Figure 1: Layered architecture: presentation, Zustand state, pure domain logic and persistence

Figure 1 shows the four layers of CloudCalc Flow. React components dispatch to a single Zustand store; the store delegates cost/performance math to pure modules and validity checks to a connection-rule table, and autosaves to localStorage.

10.2. Simulation Flow

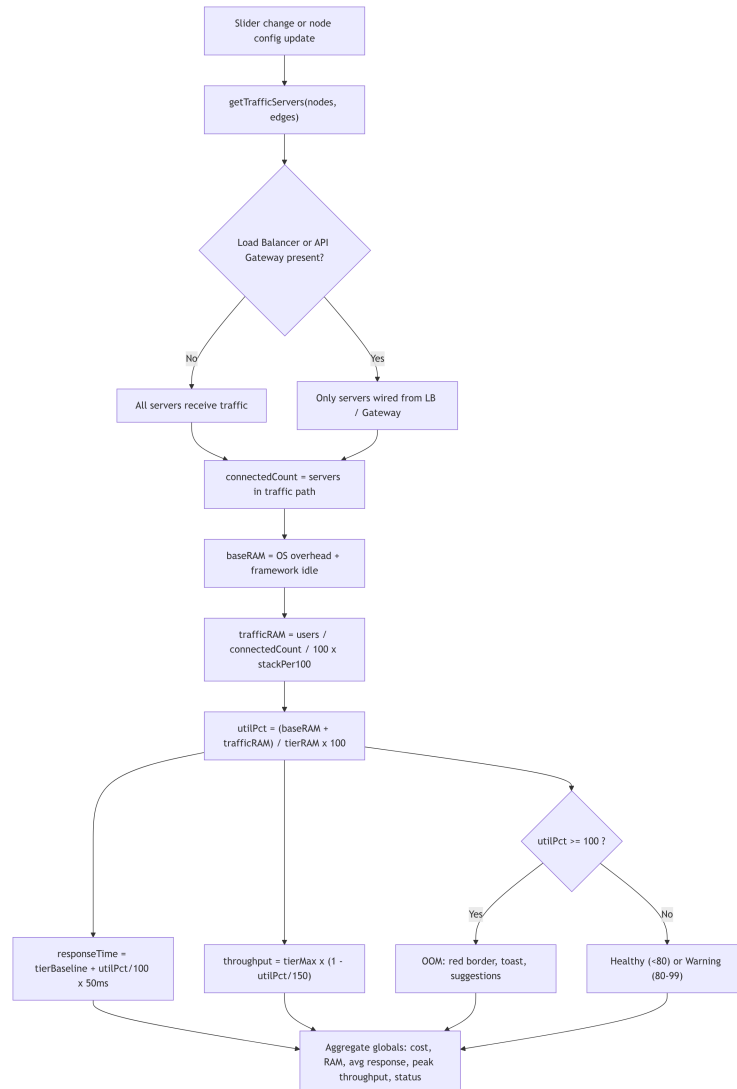


Figure 2: The calculation loop from a slider/config change to per-node status and global metrics

10.3. Data Flow

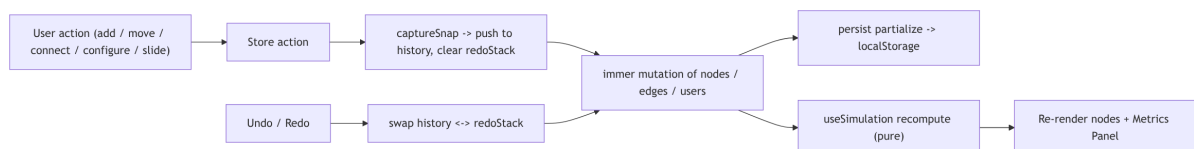


Figure 3: Action to render: history snapshot, immer mutation, persistence and pure recompute

10.4. Connection Rules

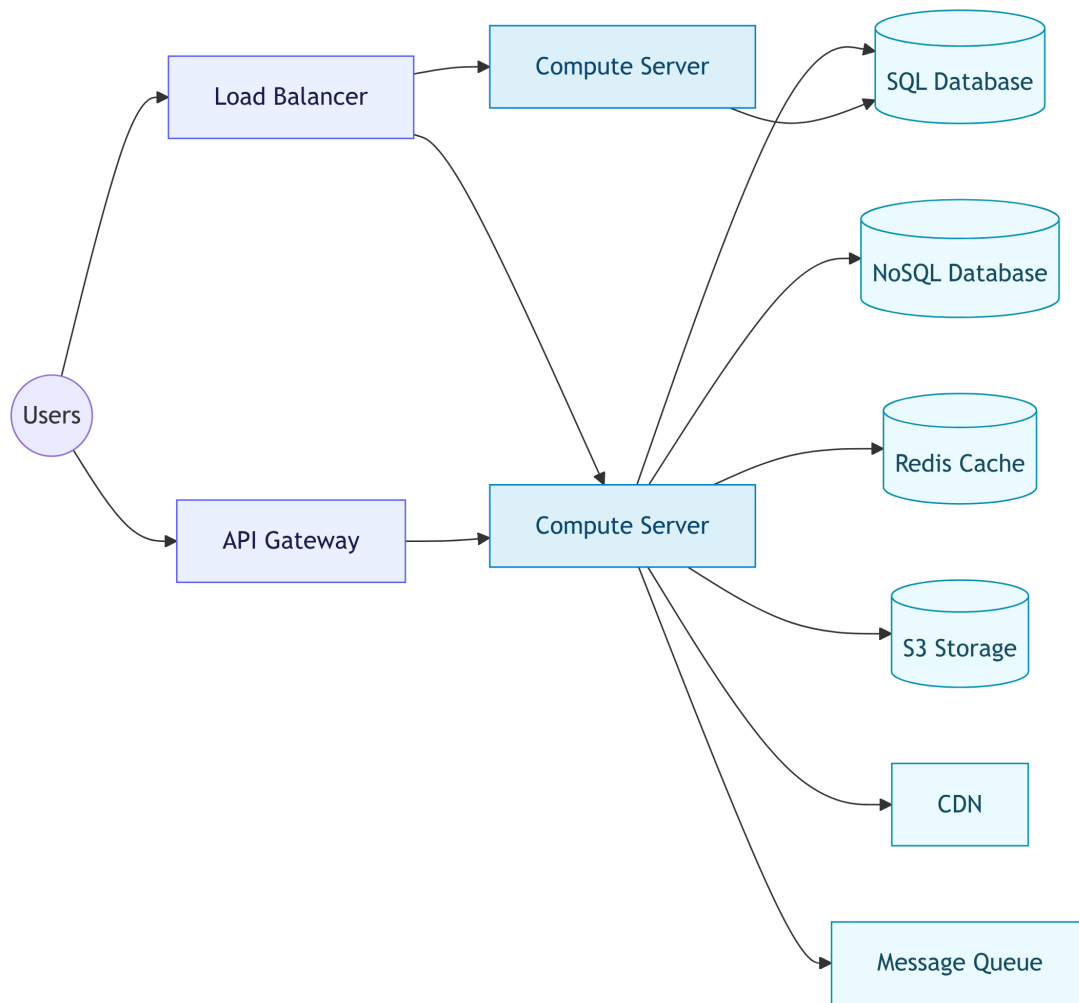


Figure 4: Enforced topology – entry points route to servers; servers write to data, cache, edge and queue sinks

10.5. Timeline (Gantt Chart)

Task	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6
Data models + simulation math + tests	✓					
Zustand store + history + connection rules		✓				
Canvas, custom nodes and edges		✓	✓			
Palette, metrics panel, config modal			✓	✓		
Warnings, suggestions, export, shortcuts				✓		
Extended components (CDN, gateway, queue, NoSQL)					✓	
Dark mode, responsiveness, verification					✓	✓

Table 6: Development timeline of CloudCalc Flow

11. User Interface

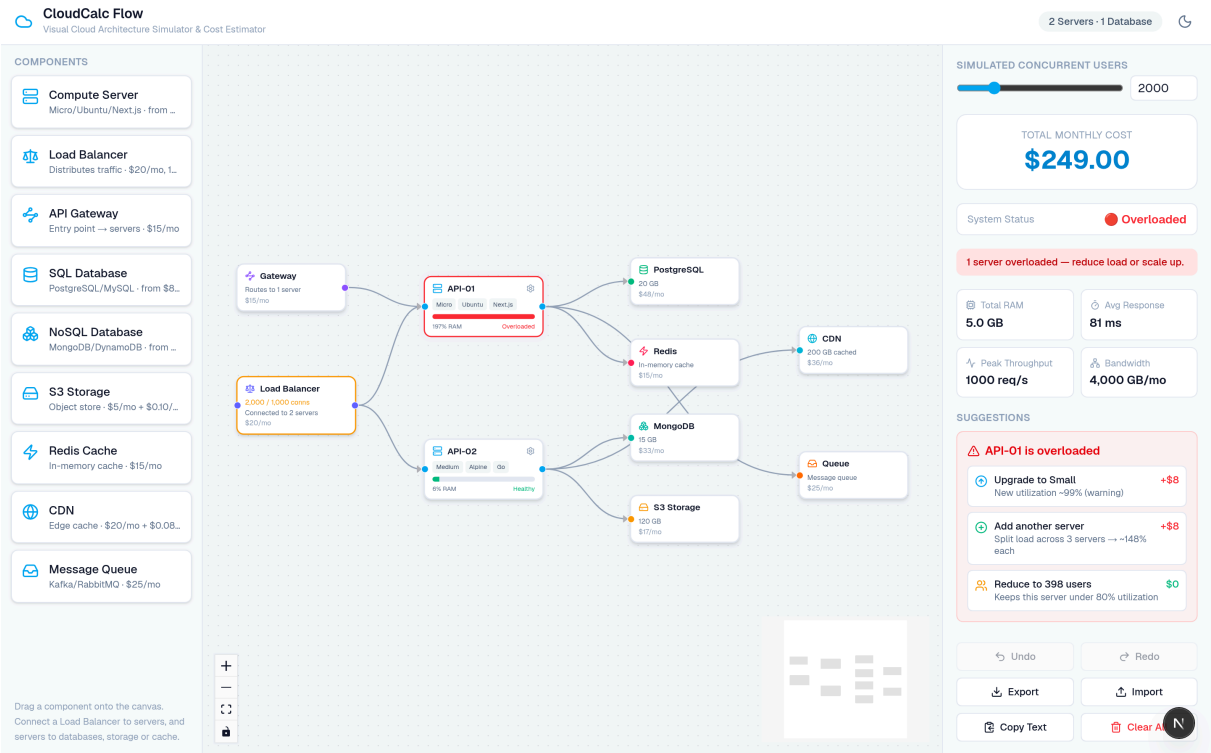


Figure 5: Populated architecture — an overloaded Micro server (red, 197% RAM) with live suggestions, alongside a healthy Medium server; total cost and system status on the right.

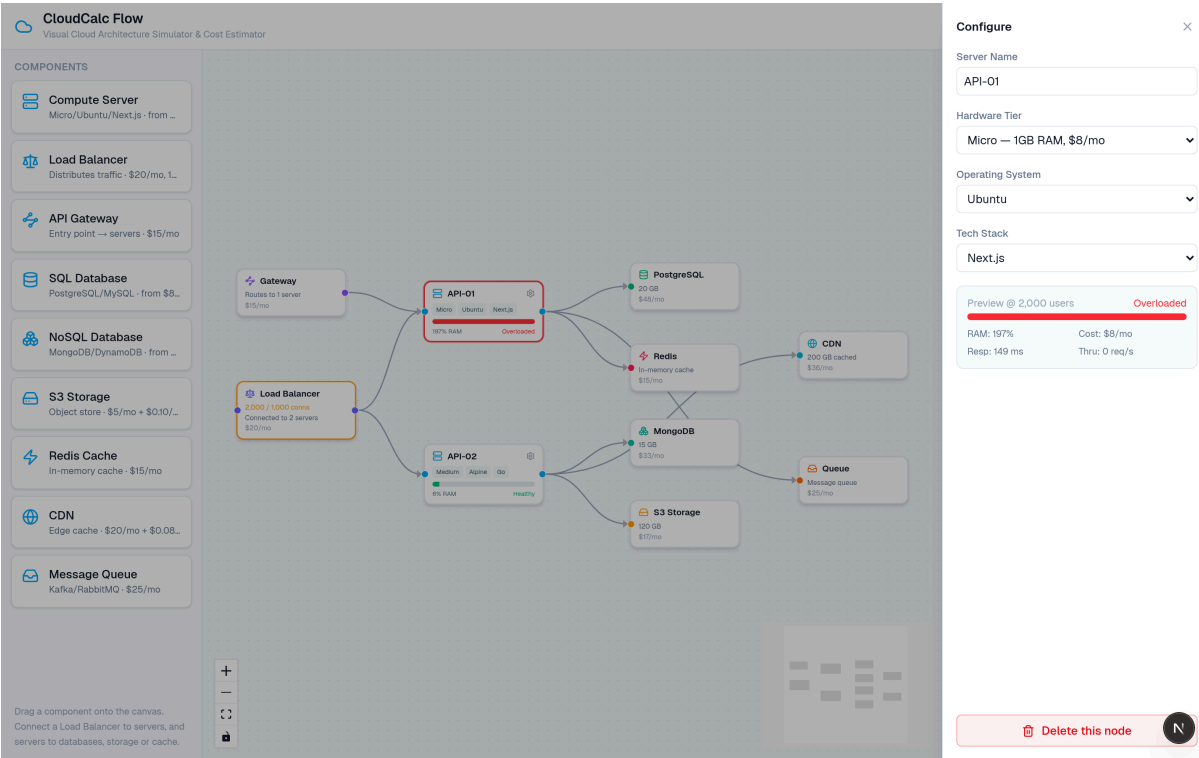


Figure 6: Node configuration panel with tier / OS / stack controls and a live preview of the resulting utilisation, cost, response time and throughput.

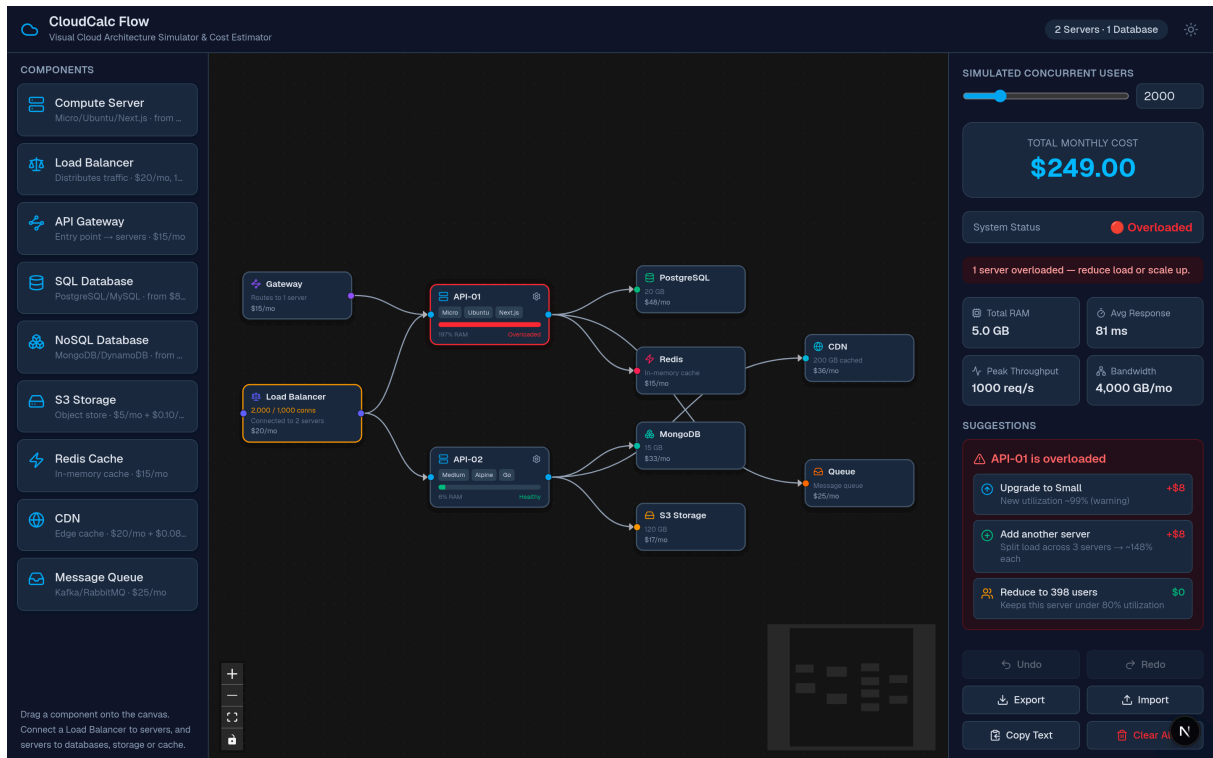


Figure 7: The same simulator in dark theme.

12. Security and Privacy

CloudCalc Flow is private by construction. It has no backend, no authentication and no network calls that carry user data: the entire architecture lives in browser memory and is autosaved only to the local device's `localStorage`. Export produces a plain JSON file the user chooses to save; import reads a local file. Because there is no server, there is no attack surface for data exfiltration, no account to breach and no telemetry. This aligns with a privacy-first, offline-capable design and sidesteps the account and data-residency concerns of hosted cost tools.

13. Future Plans

1. Provider-specific pricing profiles (AWS/GCP/Azure) selectable at runtime, replacing the single documented catalogue.
2. Preset architectures (“scalable web app”, “microservices”) and side-by-side cost comparison of two designs.
3. Richer traffic modelling — per-edge bandwidth, autoscaling policies and multi-region deployment.
4. Infrastructure-as-Code export (Terraform / Docker Compose) generated from the canvas.

14. Result

1. A fully client-side simulator was delivered with nine component types, drag-and-drop composition and enforced, type-safe connection rules.
2. A transparent per-server model computes RAM utilisation, response time and throughput live, aggregates a monthly cost, and flags overloaded nodes with cost-annotated upgrade/scale/reduce suggestions — verified end-to-end (see Figure 5).

3. Undo/redo, localStorage autosave, JSON export/import, copy-as-text, keyboard shortcuts, light/dark theming and responsive layout were all implemented.
4. The simulation math is covered by passing unit tests, and the production build compiles cleanly.

15. Conclusion

CloudCalc Flow demonstrates that architecture visualisation and cost/performance reasoning can be unified in a single, instant, no-login web tool. By pairing a node-based canvas with a small, transparent and unit-tested simulation model, it turns an otherwise static diagram into an interactive capacity-and-budget sandbox. The design keeps all data on the device, exposes every constant for teaching, and remains extensible toward real provider pricing and infrastructure-as-code export. It is both a usable planning aid and a compact showcase of a modern React/Next.js, state-management and data-visualisation skill set.

16. References

- [1] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [2] J. D. C. Little, “A Proof for the Queueing Formula: $L = \lambda W$ ”, *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961, doi: 10.1287/opre.9.3.383.
- [3] D. A. Menascé and V. A. F. Almeida, *Capacity Planning for Web Services: Metrics, Models, and Methods*. Prentice Hall, 2002.
- [4] M. Armbrust *et al.*, “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010, doi: 10.1145/1721654.1721672.
- [5] FinOps Foundation, “What is FinOps? Cloud Financial Management Framework.” [Online]. Available: <https://www.finops.org/introduction/what-is-finops/>
- [6] Amazon Web Services, “AWS Pricing Calculator.” [Online]. Available: <https://calculator.aws/>
- [7] Google Cloud, “Google Cloud Pricing Calculator.” [Online]. Available: <https://cloud.google.com/products/calculator>
- [8] Microsoft Azure, “Azure Pricing Calculator.” [Online]. Available: <https://azure.microsoft.com/en-us/pricing/calculator/>
- [9] Datadog, “Cloudcraft: Visualize your cloud architecture.” [Online]. Available: <https://www.cloudcraft.co/>
- [10] xyflow, “React Flow: Customizable library for node-based editors and diagrams.” [Online]. Available: <https://reactflow.dev/>

THE END